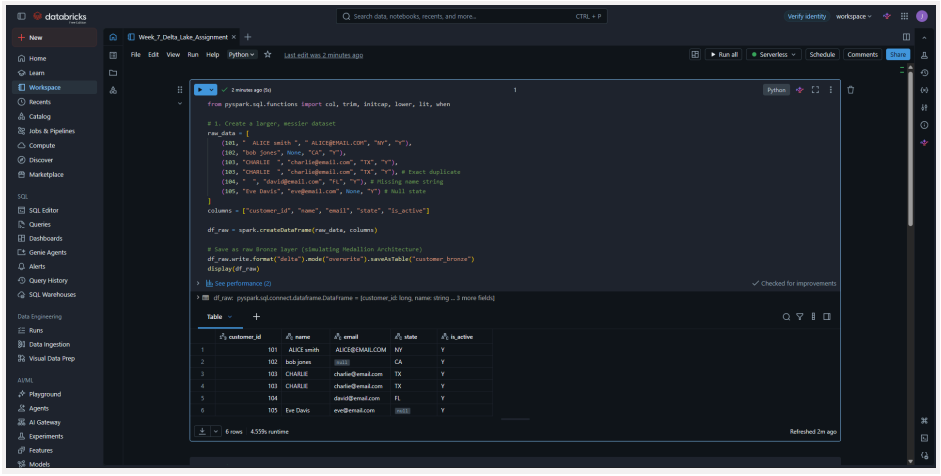
Report

Objective:
To design and implement a robust data ingestion and cleaning pipeline using Apache Spark, culminating in a Delta Lake MERGE operation to handle incremental data updates.

1. Data Loading (Bronze Layer)

To begin the pipeline, a raw, unstructured dataset was generated to represent incoming source records. This dataset intentionally included common data quality issues such as leading and trailing whitespaces, inconsistent capitalization, missing string values, nulls, and duplicate records.

The dataset was saved into the Databricks default Delta Lake format. Physically, a Delta table consists of underlying Parquet files paired with a `_delta_log` directory of JSON files. This transaction log enables ACID transactions and reliable concurrent writes, which a plain Parquet or Hive table cannot support. In the context of a Medallion Architecture, this raw, unaltered data acts as the Bronze layer, preserving the original state for lineage and auditing.

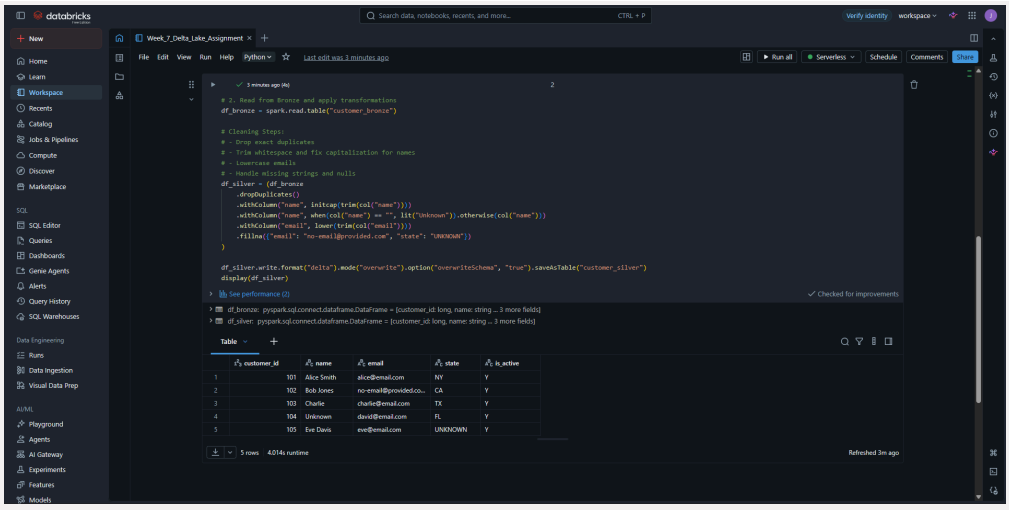


2. Data Cleaning & Structuring (Silver Layer)

Following the multi-hop architecture pattern, the data was progressively refined from the Bronze layer into a cleaner, enterprise-wide Silver layer.

The following PySpark transformations were applied to structure the data:

- **Deduplication:** Dropped identical rows to ensure data integrity.
- **String Manipulation:** Applied `trim()` and `initcap()` to standardize names, and `lower()` to standardize email addresses.
- **Null Handling:** Replaced empty strings with a default "Unknown" string and used `.fillna()` to provide default values for missing emails and states.

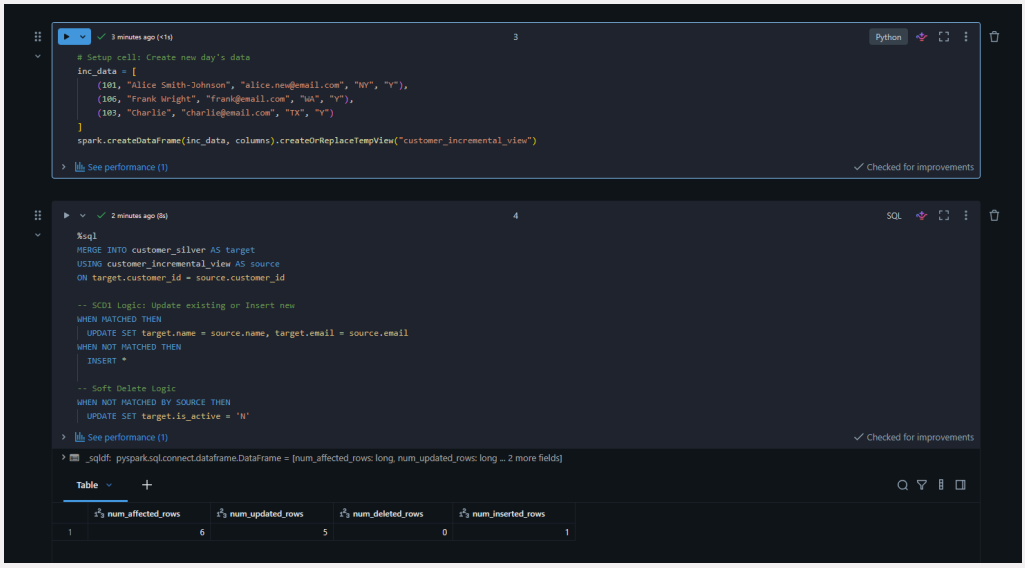


3. Incremental Upserts (SCD1) & The MERGE Operation

To test the pipeline's ability to handle evolving data over time, a second dataset (customer_incremental) was introduced representing a new day's data.

A SQL MERGE statement was used to upsert this incremental data into the target Silver table. The operation relied on up to three distinct matching clauses to determine the outcome of each row:

- **WHEN MATCHED:** Implemented Slowly Changing Dimension Type 1 (SCD1) logic by updating the target row's email and name with the new values from the source, intentionally overwriting the old state with no history kept.
- **WHEN NOT MATCHED:** Inserted entirely new customer records that only existed in the source data.



4. Historical Preservation via Soft Deletes

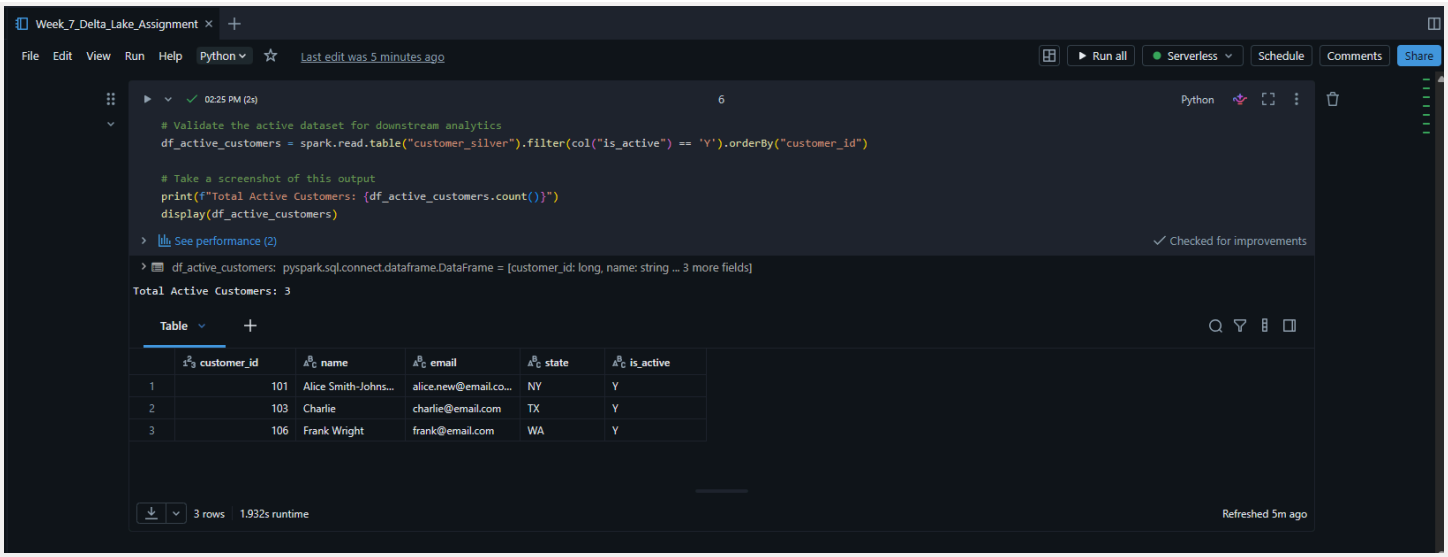
Instead of executing a hard DELETE for records that dropped out of the source system, the MERGE operation utilized the third matching clause.

- **WHEN NOT MATCHED BY SOURCE:** Handled keys that were present in the target but absent from the source.

This clause was used to implement a soft delete by updating an is_active flag to 'N' instead of physically erasing the row. This approach preserves historical data and ensures past records remain fully queryable for auditing purposes.

5. Validation and Final Output

To confirm the success of the pipeline, the dataset was queried to validate the downstream impact. The current, active state of the table can easily be accessed by analysts simply by filtering for `is_active = 'Y'`.



The final dataset accurately reflects the updated information for existing keys, the insertion of the new keys, and the successful application of the 'N' flag for stale records.